



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Concevoir l'application en utilisant UML

UML – Chapitre 6 – Classes et objets

Objectifs

Modéliser les classes utiles à l'application :

- ▶ Objets
- ▶ Classes
- ▶ Attributs et opérations
- ▶ Associations



Le **diagramme de classes** est le point central dans un développement orienté objet

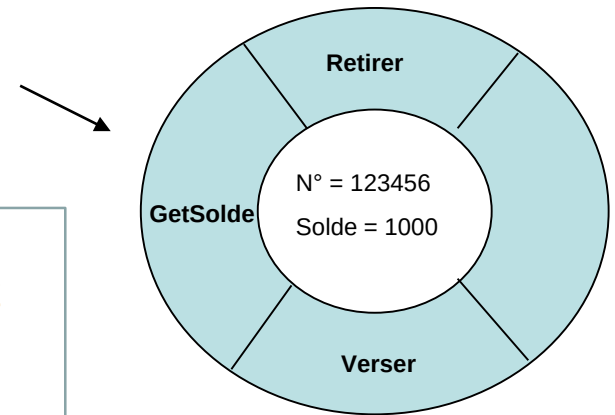
Qu'est-ce qu'un objet ?

- ▶ Un objet est une **instance d'une classe**
 - La **classe** est le **moule/patron** pour créer des objets
- ▶ Un objet a un **état "ce qu'il sait sur lui"**
 - chaque élément d'information est appelé **attribut**
- ▶ Un objet a un **comportement "ce qu'il sait faire"**
 - chaque comportement est appelé **opération**
- ▶ Un objet a un **cycle de vie** :
il naît, il vit, il meurt

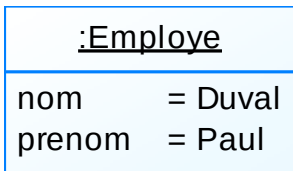
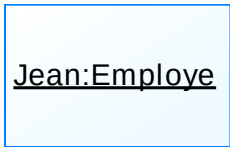


Boîte noire :

Celui qui utilise un objet connaît seulement son interface. Il n'a pas besoin de savoir comment il est implémenté



Représenter des objets



► 2 compartiments :

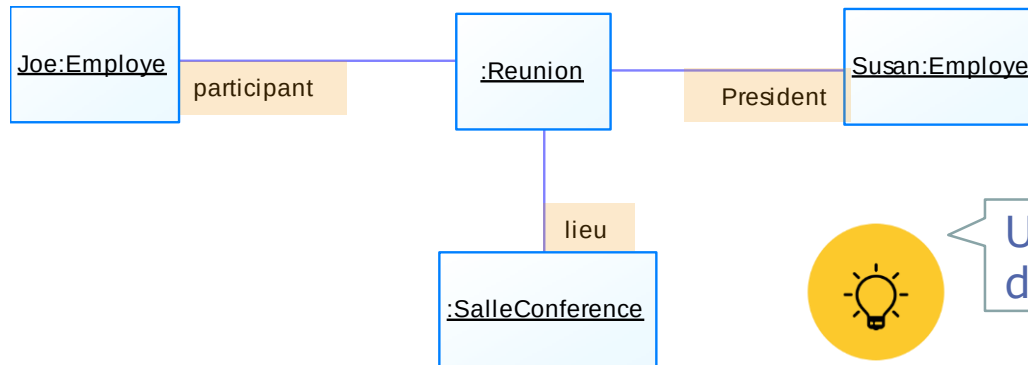
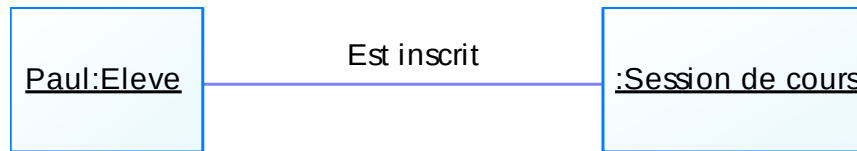
- Nom de l'objet et/ou nom de la classe précédé de ":"
- Optionnellement, attributs et leur valeur
- Les opérations ne sont pas montrées ce sont les mêmes pour tous les objets d'une même classe



Le nom est souligné pour indiquer qu'il s'agit d'un objet

Représenter des liens entre objets

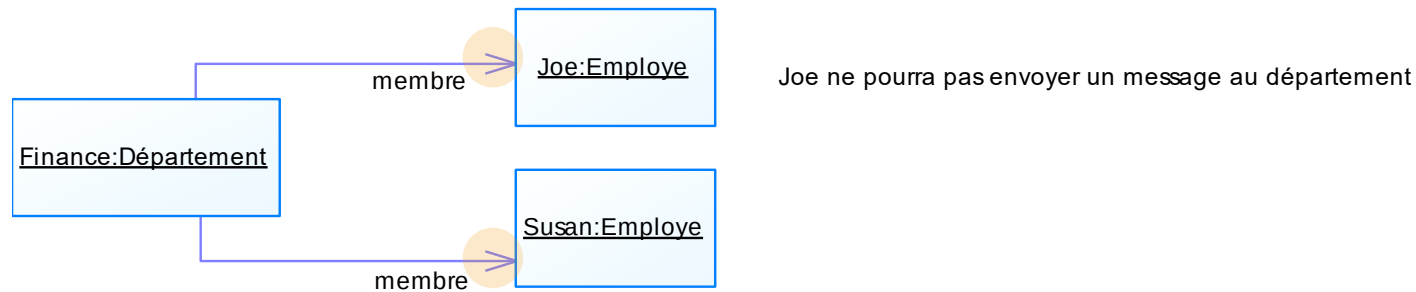
- ▶ Pour collaborer les objets ont besoin de connaître d'autres objets
- ▶ Un objet pourra envoyer un **message** à un autre objet : appeler une de ses opérations



Un objet peut avoir un rôle dans une association

Navigabilité

- ▶ Par défaut, un lien est **bidirectionnel**, **navigable** dans **les 2 sens**
- ▶ Afin de diminuer le **couplage** entre objets, on peut réduire la navigabilité



La flèche indique que des messages ne peuvent être envoyés que dans un sens

Pourquoi représenter des objets ?

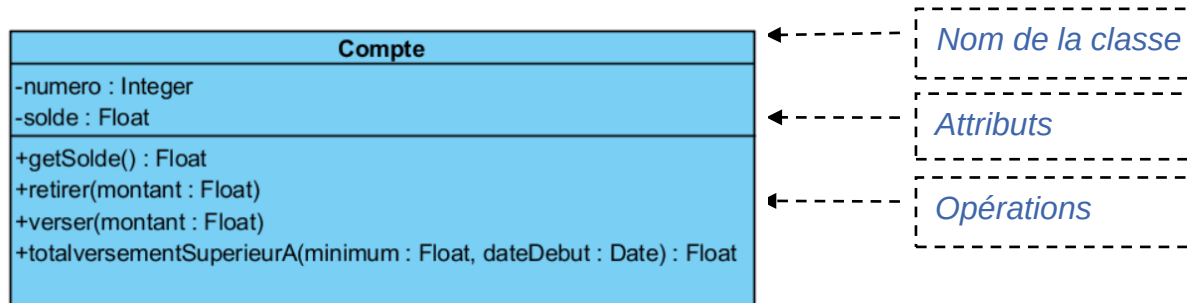
- ▶ Un diagramme d'objets montre des objets spécifiques et leurs liens à un instant donné
 - Pour examiner un cas intéressant ou affiner un aspect délicat
 - Pour représenter des cas de tests
 - On peut en faire un par cas d'utilisation
- ▶ Un diagramme d'objets est une instance du diagramme de classes

Qu'est-ce qu'une classe ?

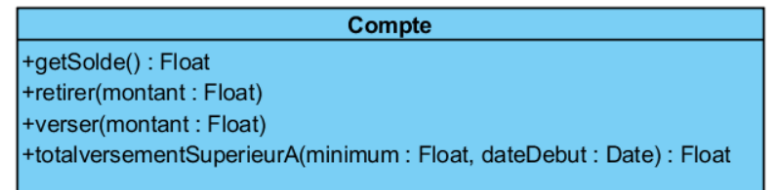
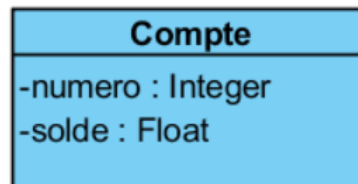
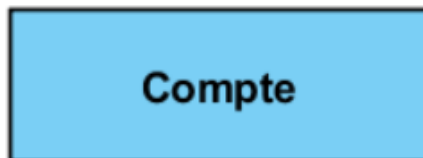
- ▶ Le moule/patron à partir duquel les objets sont instanciés
- ▶ Une description abstraite d'un ensemble d'objets ayant les mêmes caractéristiques
- ▶ Une classe est un type décrit par :
 - Un nom de classe
 - Un ensemble d'attributs décrivant l'état d'une instance
 - Un ensemble d'opérations décrivant le comportement des objets
- ▶ Des exemples : Personne, Compte

Représenter une classe

► 3 compartiments standards



► Chacun des compartiments peut être omis Pour autant, cela ne signifie pas qu'il est vide



Zoom sur les attributs

- ▶ Un attribut est décrit de la manière suivante :

visibilité nom [multiplicité]: type = valeur_initiale

- ▶ Visibilité

- *private*

- + *public*

- # *protected*

- ▶ Encapsuler les données pour les protéger

⇒ les attributs doivent être privés

Attribut ou opération de classe

- ▶ Par défaut, les attributs et les méthodes ont une **portée objet** ou instance
 - Chaque **objet** a **sa propre copie d'un attribut**
 - Une opération s'applique à un objet particulier
- ▶ Certains attributs ou méthodes ont une **portée classe** (*static en C# ou java*)

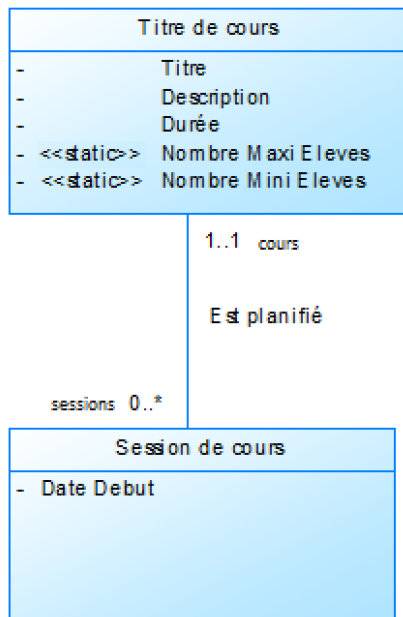
Titre de cours	
-	Titre
-	Description
-	Durée
- <<static>>	Nombre Maxi Eleves
- <<static>>	Nombre Mini Eleves
+ <<static>>	GetNbMiniEleves()

- ▶ Une association est une **relation sémantique** durable entre classes
 - Un élève **s'inscrit** à une session de cours
 - Une personne **possède** plusieurs voitures

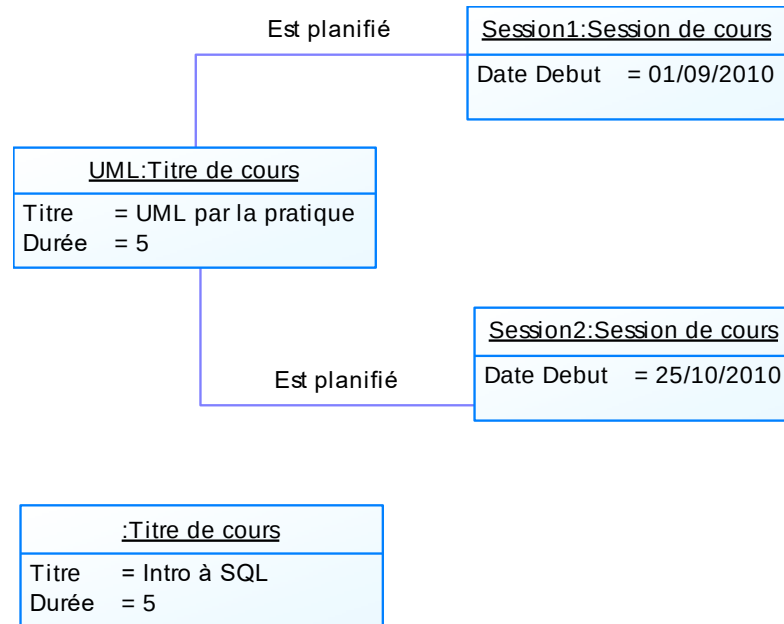
- ▶ Une classe ou ses instances peuvent participer à plusieurs types de **relations**
 - Association
 - Agrégation et composition
 - Généralisation/spécialisation

Association

- Une association est une relation entre classes qui décrit les liens possibles entre les instances



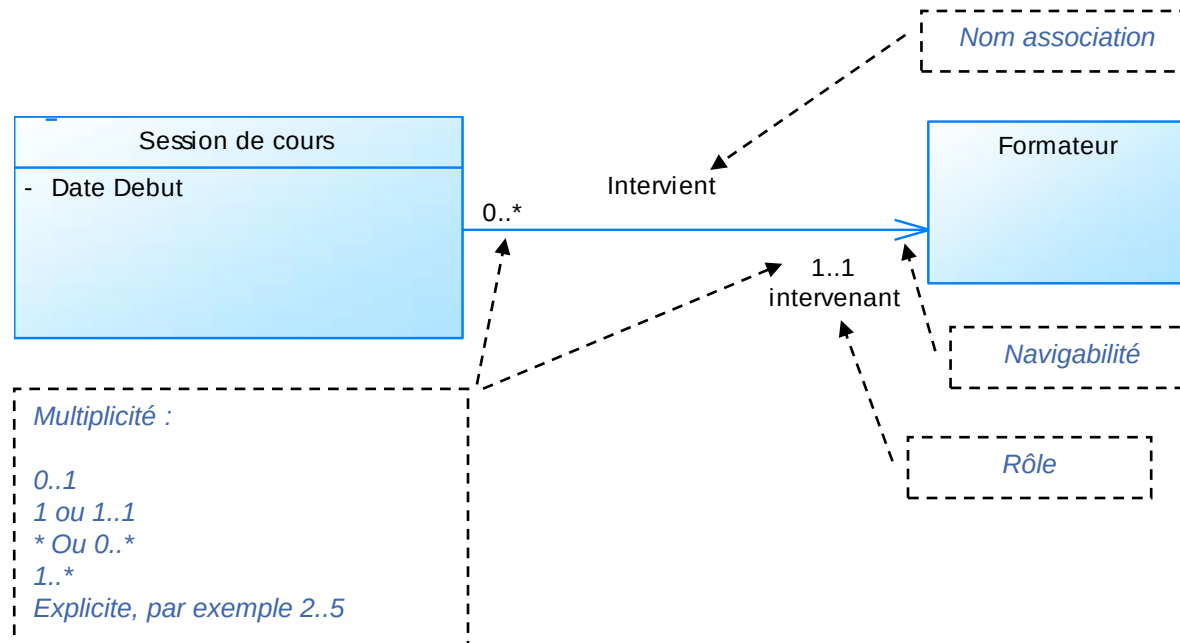
Le diagramme de classes



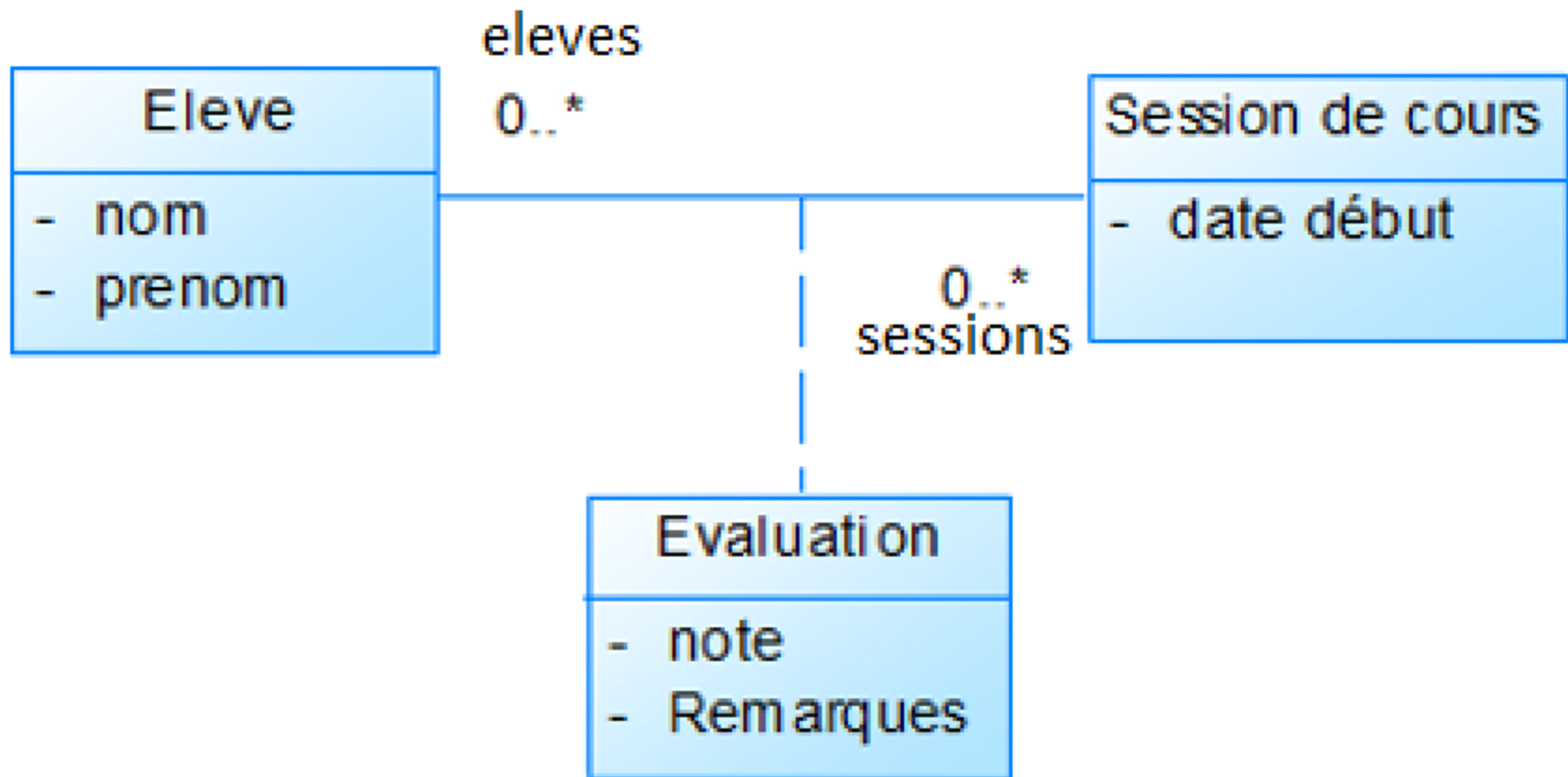
Un diagramme d'objets

Association

- Une association peut être décorée

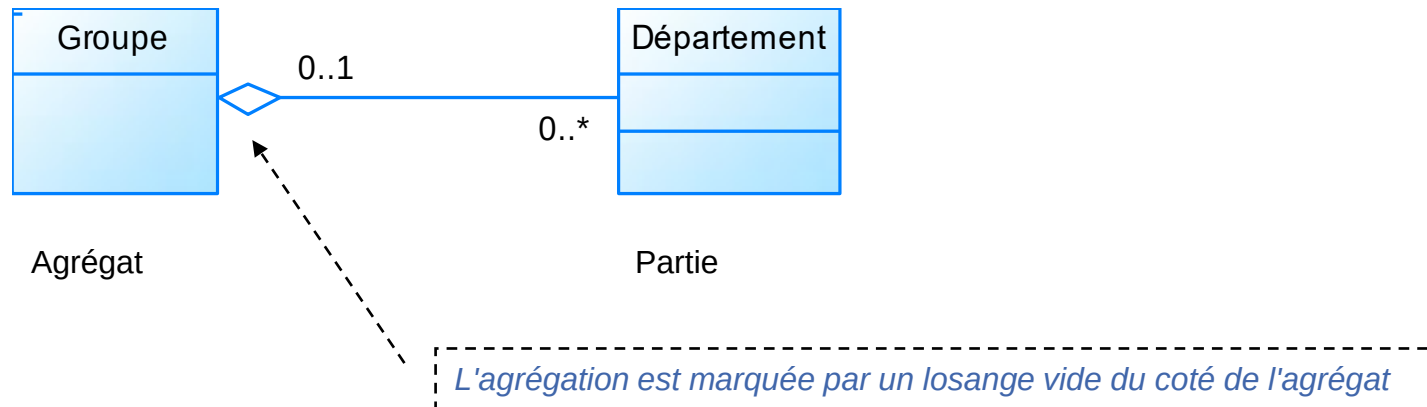


Classe d'association



Agrégation

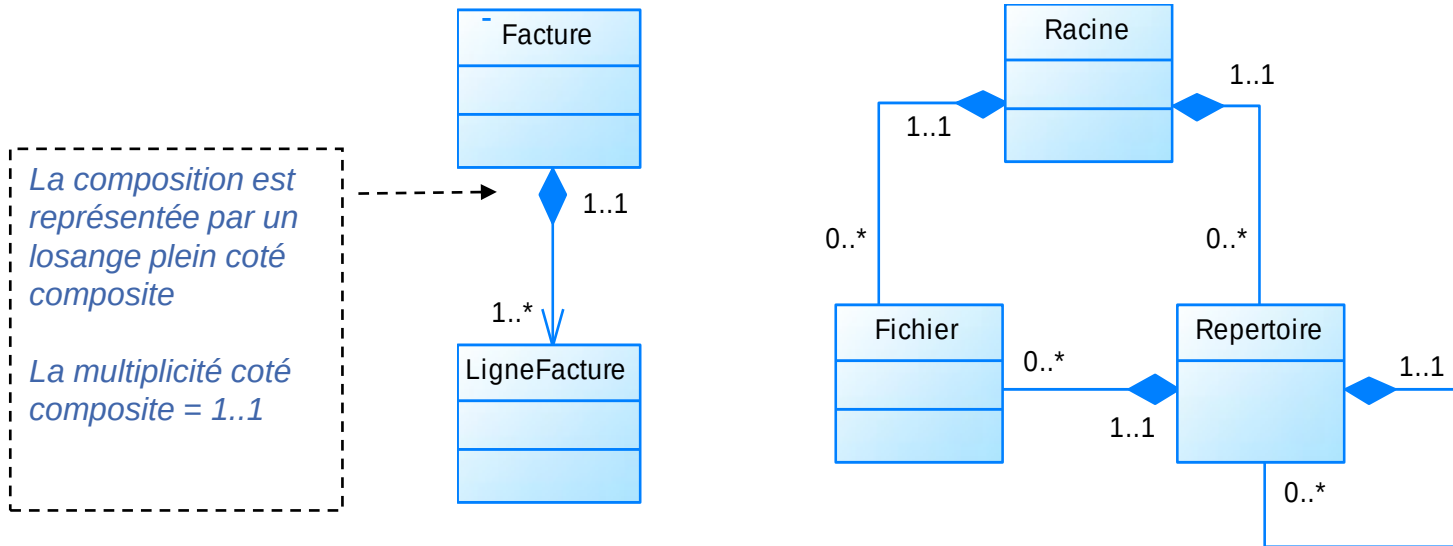
- ▶ Une forme d'**association** qui représente une relation *"est une partie"*
 - Une partie peut exister sans être reliée à un agrégat
 - Une partie peut être reliée à plusieurs agrégats
- ▶ Une agrégation est une relation **antisymétrique** et **transitive**



Composition

► Une forme plus forte d'agrégation

- Une partie ne peut exister en dehors d'un agrégat
- Une partie ne peut appartenir à plus d'un agrégat
- Toutes les parties sont détruites quand l'agrégat est détruit



Association // agrégation

► Les questions à se poser :

- ? X est partie de Y → **agrégation**
- ? Les objets reliés sont plutôt indépendants les uns des autres → **association**
- ? Des opérations du tout s'appliquent automatiquement aux parties ? → **agrégation**
- ? Des valeurs d'attributs du tout se propagent vers les parties ? → **agrégation**

Dans le doute → **association**

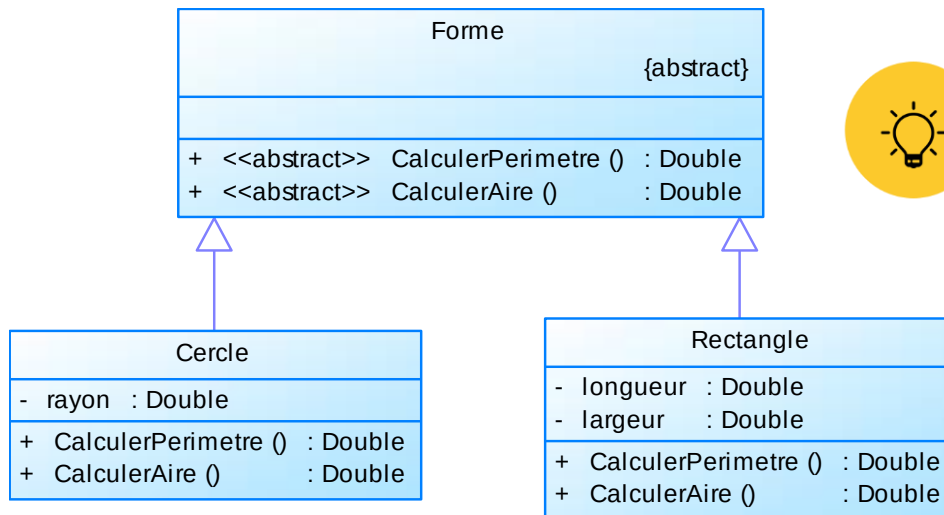
Généralisation-spécialisation

- ▶ La sous-classe hérite de tous les attributs, relations, méthodes de la classe parente
 - Les attributs et opérations de la super classe ne peuvent pas être cachés/enlevés dans les sous-classes
 - La sous-classe peut avoir des attributs ou des méthodes spécifiques
 - La sous-classe peut redéfinir une opération héritée



Classe abstraite

- Une classe peut spécifier une opération sans l'écrire
 - La méthode est abstraite
 - La classe est abstraite
- Une classe abstraite ne peut pas être instanciée

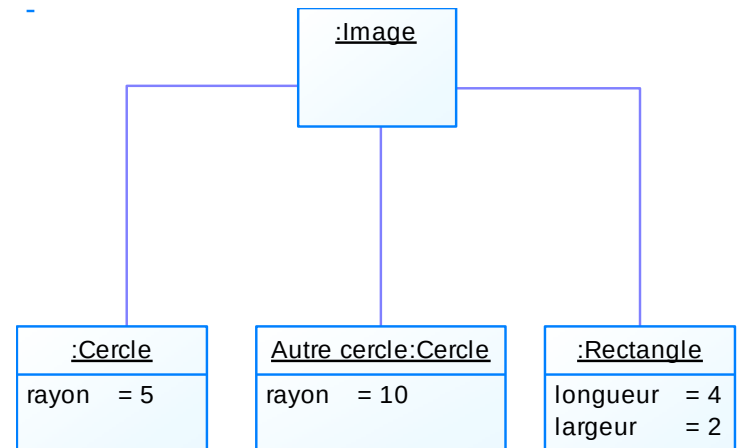
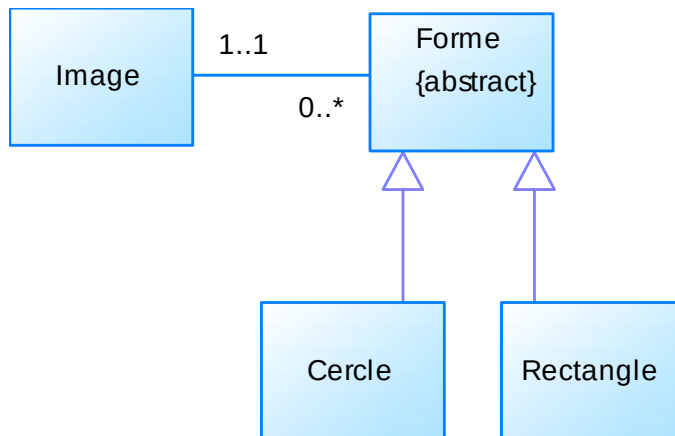


Toute Forme doit pouvoir calculer son périmètre et son aire

Seules les sous-classes savent comment le faire

Principe de substitution

- ▶ Partout où une instance d'une classe est attendue on peut fournir une instance d'une sous-classe
- ▶ Permet de constituer des collections hétérogènes



► Attribut ou Classe ?

- Si on peut demander à un élément seulement sa valeur
⇒ c'est un attribut
- Si on peut demander à un élément d'exécuter des actions ou s'il existe des liens vers d'autres classes
⇒ c'est une classe

► Ne pas abuser de la généralisation !

- La sous classe doit être conforme à 100% aux spécifications de la superclasse

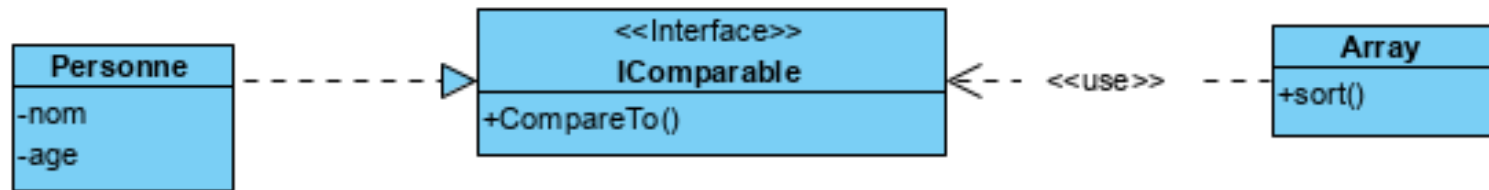
► Agrégation ou composition ?

- Composition ⇒ multiplicité = 1 + le cycle de vie des parties dépend du cycle de vie de l'agregat

Interface

- ▶ Une sorte de classe abstraite "pure" dans laquelle aucune méthode n'est implémentée
- ▶ Une interface décrit un contrat : les opérations attendues
- ▶ Une classe qui réalise une interface doit implémenter toutes les opérations du contrat
- ▶ Une classe hérite au plus d'une classe mais peut implémenter plusieurs interfaces
- ▶ Les interfaces permettent de séparer les opérations de leur mise en œuvre

Interface



- ▶ Une classe qui utilise une interface
 - N'a accès qu'aux opérations décrites par l'interface
 - Est reliée à l'interface par une **relation de dépendance**
- ▶ Les interfaces **réduisent la complexité vue du client**
- ▶ Les interfaces étendent la sécurité du système

Quand modéliser les classes ?

- La démarche est itérative, le diagramme de classes est affiné à chaque étape :

